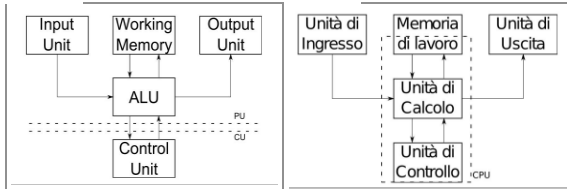
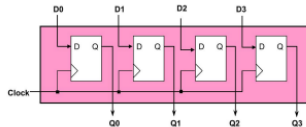


Andiamo ora a vedere il processore Z64 : ricordiamoci che questo processore si basa su architettura di von Neumann . Questo processore implementa un sotto insieme delle funzionalità dei processori intel . Composto da circuiti pilotabili tramite sw . **L'ISA (instruction set architecture)** di un processore definisce tutto quello che può fare e come lo fa. Esistono due famiglie RISC e CISC: nella prima si ha un insieme ridotto di istruzioni ,programmi più lunghi , più efficiente dal punto di vista energetico e di più semplice progettazione . Inversamente ai CISC , ci sono i RISC : aumenta la quantità di istruzioni , programmi più compatti , aumento dal punto di vista energetico ed aumenta la complessità della progettazione (processore più grande e circuiteria annessa) . Per quanto riguarda l'architettura , la dobbiamo rivisitare , in quanto se fosse quella tradizionale ed avendo che il processore è molto veloce rispetto ai registri, portiamo un pezzo di memoria di lavoro all'interno del processore . Quindi in sintesi :



Per quanto detto finora , nel processore abbiamo detto che ci va un pezzo della memoria di lavoro , e questo pezzo della memoria di lavoro è costituita da **registri** : blocchetti di memoria di 1 kb , i quali sono organizzati a parole binarie . Sono realizzati tramite FF (unico circuito che immagazzina un bit in memoria) : vengono messi in cascata un numero di FF:

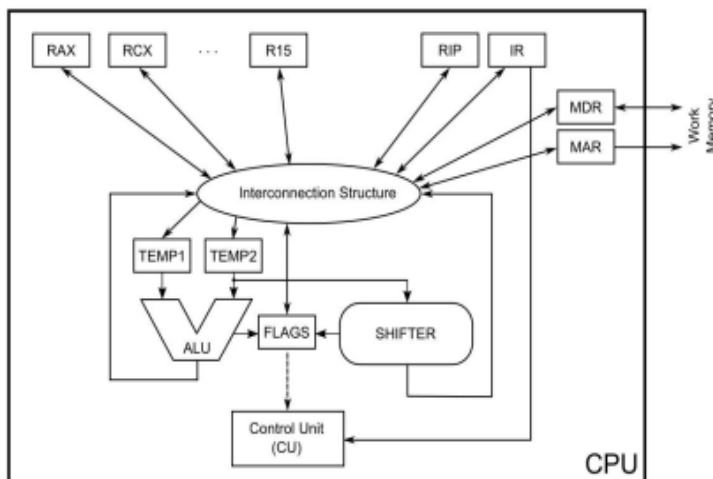


La quale velocità viene decisa dal clock .

I registri fanno parte dell'isa . Non tutti i registri sono esplicitamente utilizzabili : i registri si dividono in :

1. Fondamentali
 - a. Necessari per soddisfare architettura di un processore
2. Visibili al programmatore
 - a. Il programmatore può interagirvi per letture/scritture dati
3. Invisibili al programmatore
 - a. Il programmatore non può leggere/scrivere: sono le istruzioni che vanno a modificare questi registri.

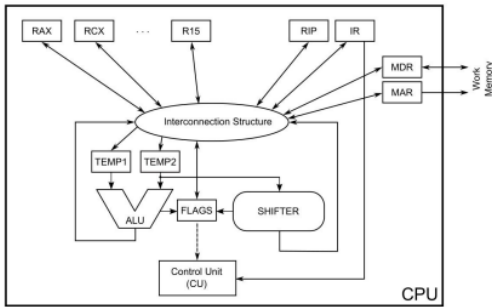
L'architettura dello z-64 è la seguente:



In dettaglio : i registri RAX,RXB ,...R15 sono 16 registri a 64 bit che possiamo usare : in realtà ne abbiamo 14 da usare. MAR,MDR sono registri tampone che costituiscono interfaccia verso il mondo esterno : velocità molto alte e costituiscono buffer tra 2 reti sequenziali a velocità diversa. RIP (instruction pointer) importante registro che ricorda al processore l'ultima sequenza delle istruzioni : tiene traccia del programma . ALU+Shifter reti iterative che ci permettono di eseguire operazioni su parole ad n bit : nel caso di quelle veloci servono anche i bit di stato , i quali vengono immagazzinati nel registro flags (FF) . Non dimentichiamoci della struttura di interconnessione (fili che fanno viaggiare le istruzioni) . L'unità di controllo è (gigante) rete sequenziale che pilota il nostro hardware „genera i bit di controllo che vanno verso tutti i componenti per configurarli in modo corretto. Vediamo ora come mantenere i dati stabili (vedasi discorso del tempo di propagazione) tra un colpo di clock ed il successivo : usiamo il registro IR (instruction register e registro tampone) il quale mantiene la codifica binaria dell'istruzione che il processore esegue . Vediamo ora come vengono processate le istruzioni :

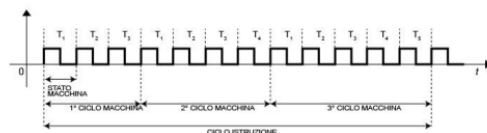
1. **Fetch**
 - a. Vado in memoria a recuperare la codifica binaria della mia istruzione macchina da eseguire e la metto nel registro IR . Conversione/codifica input/output fatta dal processore
2. **Decode**
 - a. Input dal registro IR viene interpretato in sequenza
3. **Execute**

- a. Traduzione istruzione macchina in istruzioni interne , quindi esecuzione sequenza passi per eseguire istruzione.

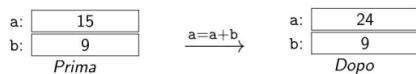


Questo modello è abbastanza datato in quanto è un modello **uni ciclo** : organizzazione interna dei processori nel quale ogni istruzione viene eseguita in un colpo di clock, il quale viene preso minimo in base al critical path (per la stabilizzazione) e se operazioni differenti non posso riciclare hw , quindi aumenta il numero di componenti , mentre per quelle **multi ciclo** : rompo il circuito in più parti, accorcio il tempo di clock : un'istruzione è eseguita in più colpi di clock , inoltre le componenti possono essere riusate tra un ciclo ed il successivo . Con questo modello si può avere il riciclo dell'hw . Da notare che nella fase di esecuzione , si eseguono più istruzioni : quindi la fase di execute si spezza in sotto-fasi , quindi per eseguire un'istruzione :

- **Ciclo istruzione**: intervallo temporale necessario ad eseguire una istruzione nella sua interezza
- **Ciclo macchina**: intervallo temporale necessario ad eseguire una fase (fetch, decode, execute)
 - A seconda del tipo di istruzione, possono essere necessari un numero diverso di cicli macchina (es: più accessi in memoria)
- **Stato macchina**: periodo di tempo necessario per stabilizzare la rete dell'unità di calcolo (corrispondente al clock)



Notiamo che non tutte le istruzioni hanno lo stesso ciclo macchina. Vediamo ora la semantica di un'operazione :



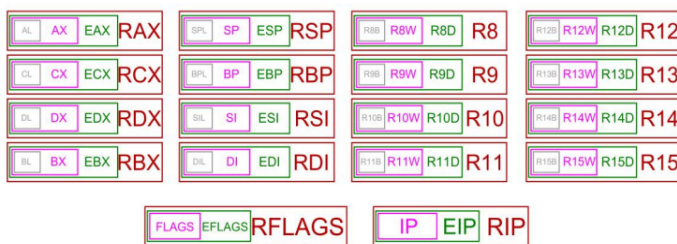
Dove a e b sono i registri : li consideriamo come due scatolette . **Quindi prima dobbiamo capire dove sono memorizzati, decidere dove scrivere il risultato , decidere quali istruzioni svolgere . Il formato è il seguente :**

MOV < sorgente> , <destinazione>

s può essere B, W, L, Q-

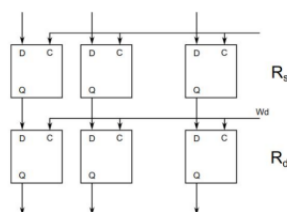
Dove byte, word, long-word, quad-word : 8-16-32-64

In dettaglio (general purpose) :



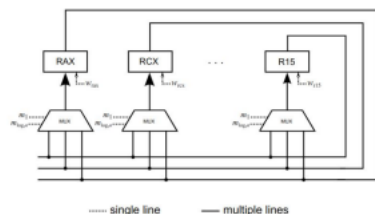
■ 64-bit Register ■ 32-bit Register ■ 16-bit Register ■ 8-bit Register

Possiamo virtualizzare i registri : uso i registri di più piccoli (da 64 si va a 32 ecc ecc) : attenzione che si corrompono i registri . L=low meno significativi , E= extended 32 bit . Da R8 a R15 sono quelli di amd , che intel li ha presi : registri veramente general purpose. Quelli in basso partono da 16 bit . Come sono realmente realizzati questi registri? Attraverso dei FF collegati tra loro , attraverso fili (il valore viene memorizzato ad ogni colpo di clock) :

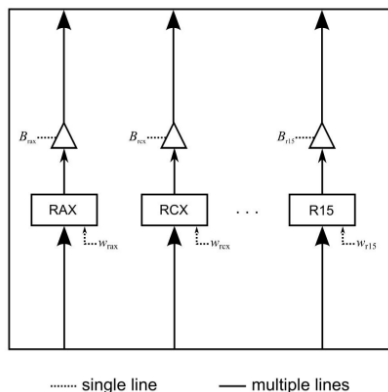


Ma notiamo che se il numero dei FF cresce , si hanno molti collegamenti , quindi possibile aumento complessità (64 fili per ogni registro e verso tutti i registri). In alternativa a questa connessione punto-

punto , lo implementiamo così : attraverso uso dei mux (in ingresso scegli uno sola configurazione) , così da 64 fili per ogni registro se ne ha uno solo :

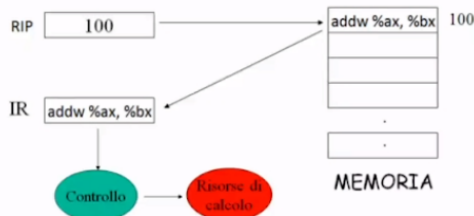


Notiamo che per ogni registro si hanno dei segnali di controllo, i quali permettono o meno la scrittura del dato nel registro , basta che siano abilitati (segnalo W_{rax}). Riassumendo : **1 canale di 64 bit per ogni registro, ad ingresso un mux che sceglie l'uscita corrispondente ed un segnale di abilitazione di lettura/scrittura**. Questi segnali vengono generati dall'unità di controllo. Problema con questo schema : costo eccessivo dei mux e delle linee di interconnessione . Si può migliorare ? Sì : **interconnessione tramite bus** : quindi i registri possono leggere e scrivere su stesso collegamento : possibile sia ricezione che trasmissione , ma c'è un controllo che permette ad un unico registro di trasmettere i dati usando due bit di controllo : Write enable per scrivere e buffer-tri state che si comporta da interruttore :

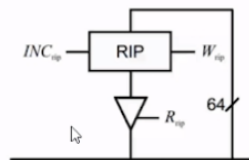


Avendo così solo 64 fili di interconnessione. Si deve separare il momento della lettura da quello della scrittura , per evitare che i dati non sono stabili. Torniamo ora ai principali due registri : IR e RIP : il primo mantiene logicamente informazione su ultima istruzione eseguita (memorizzo indirizzo prossima istruzione) , la quale è ad indirizzo prossimo : architettura Von Neumann funziona sulla sequenzialità delle istruzioni , a meno che non gli si chiede di saltare ad istruzioni "lontane". Mentre per quello che riguarda il secondo mantiene in memoria l'indirizzo della prossima istruzione da eseguire.

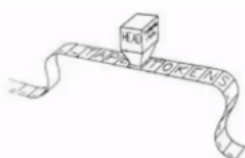
Andiamo a vedere come viene eseguita un'istruzione : fetch -> decode -> execute :



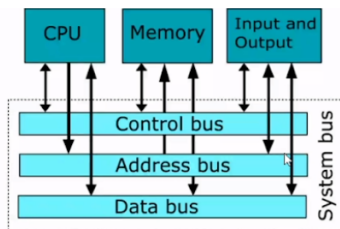
Ricordiamo che incrementare RIP di 1 significa spostare in avanti di 8 bit . Questo incremento di 8 in hardware viene realizzato così: dobbiamo fare una somma di 8 (terzo bit meno significativo) all'indirizzo precedente , quindi aumenta il numero di somme che si devono fare . Quindi usiamo dei **registri contatori** , ovvero registri che incrementano il contenuto dell'IP (usando dei Full-Adder) :



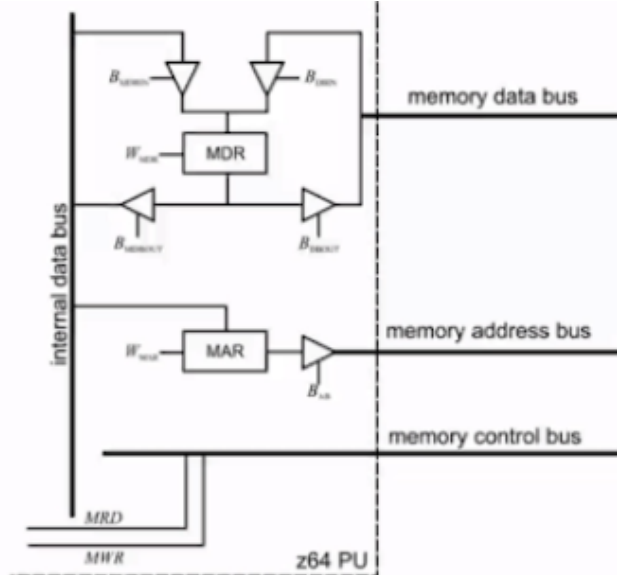
Dove INC è un segnale di controllo che permette di fare incremento del valore . Tornando ad architettura di Von Neumann , andiamo a focalizzarci sul registro MAR : è un registro che permette di scambiare informazioni tra processore e RAM , quindi nella fase di fetch prende l'indirizzo dell'istruzione da RIP . Si ha quindi che il segnale viaggia sul bus dati. Focalizziamoci sul RIP : indirizzo istruzione corrente , il processore dopo il fetch abilita il segnale di controllo : quindi RIP punta ad istruzione prossima . **Quindi RIP mantiene indirizzo istruzione corrente nella fase iniziale di fetch** . Per quanto riguarda la memoria : lo pensiamo come un nastro di n celle (contenenti i dati) , di dimensione fissata (8 bit -> 1 byte) . Ogni cella ha un numero e si parte dal numero 0 fino ad N-1 : ovvero l'indirizzo in memoria (spiazzamento all'interno del vettore) . Attenzione a contesto indirizzo e contesto dati .



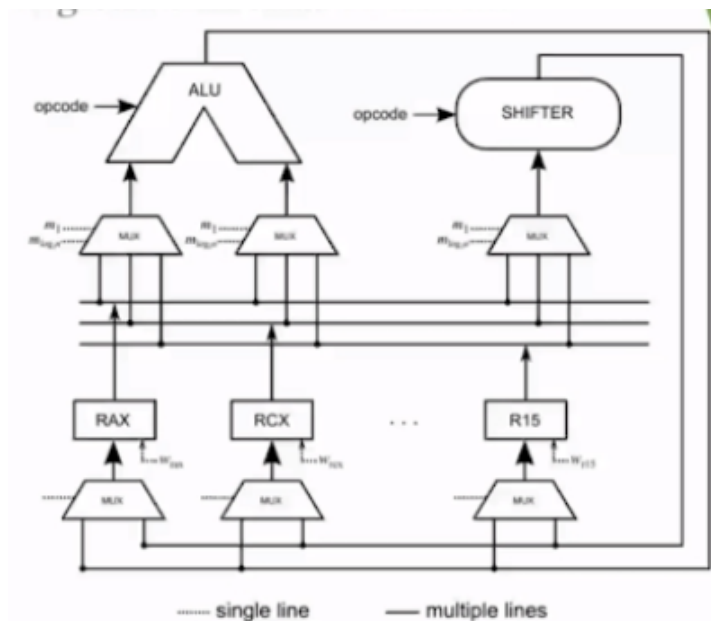
Vediamo ora come avviene lo scambio di info tra registri e processore : attraverso il **bus di sistema** (64 fili) , il quale si divide **control bus** , **address bus** e **data bus** :



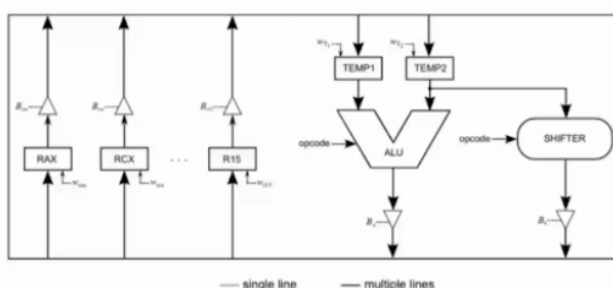
I primi sono fili che permettono comunicazione CPU - memoria . Nel secondo invece anche se si parla di architetture a 64 bit , si fermano a 48 bit (2^{48} bit) . Vediamo ora come è realizzato :



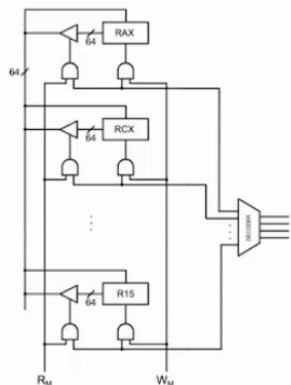
Notiamo il Wmar (write enable mar) : segnale di controllo che permette la scrittura del dato in memoria . Per quanto riguarda invece il buffer-tri-state si comporta come interruttore . Analogamente per MDR che ne ha 4 di buffer-tri-state (due in ingresso, due in uscita) . **Questa struttura di interconnessione, non è completa** in quanto manca quella che permette l'interconnessione tra registri e circuiti di calcolo :



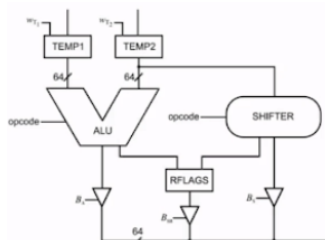
Con internal data bus non si riesce a trasferire dati (in modo stabile) verso ALU , quindi introduco un canale dedicato per questa comunicazione : ma c'è un problema : 16 mux , segnali di controllo aumentati , stesso registro scritto da alu e shifter !! Come risolviamo ? Uso registri tampone (2) : vengono collegati al bus dati interno :



Attenzione alla fase nella quale la ALU fa operazioni (calcola dati errati) : o aggiungo opcode ad Alu oppure ignoro totalmente output , grazie al buffer-tri-state, il quale non permette di propagare queste informazioni errate . Per ogni registro general-purpose ci sono 2 segnali di controllo (W/R) , avendo così 32 fili che escono da unità di controllo ed alu è ASF , quindi 32 segnali di ingresso : riorganizziamoli (disaccoppiamo W/R dai dati) : uniche due operazioni che posso eseguire sono W/R , quindi il numero dei registri scende a $16 : 4$ segnali di controllo . Quindi in generale : da funzione che uscita a 32 bit a funzione che ha in uscita 6 bit (4 controllo + W + R) . Arriviamo così al **banco dei registri** : selezione un solo registro per eseguire un'operazione di W/R :



Ogni registro viene codificato in binario (4 bit) , ed inoltre hanno in ingresso due segnali di controllo (Rm e Wm) , i quali vanno in and con l'uscita del decoder . Disaccoppiamento una linea di selezione registro da segnale di controllo che dice quale operazione eseguire su quel registro. Torniamo a parlare del **registro Flags** : registro che mantiene i bit di stato in uscita dalle reti iterative (shift ed ALU) e li memorizza.:



I dati vengono estrapolati dal registro flags usando la connessione con il data bus interno , la quale comunicazione (permesso/negazione) viene gestita dal buffer-tri-state. Com'è organizzato?



I bit di stato sono aggiornati da ALU e shifter per memorizzare le informazioni riguardanti l'ultima operazione eseguita , mentre quelli di controllo sono modificabili da programmatore per modificare le funzionalità del processore . C'è un bit settato ad 1 (posizione 1) , il quale bit verrà usato e collegato opportunamente per "creare"/interagire con valori costanti (costanti) . In dettaglio , **ma attenzione al contesto** :

- **carry (CF)**: vale 1 se l'ultima operazione ha prodotto un riporto
- **parity (PF)**: vale 1 se nel risultato dell'ultima operazione c'è un numero pari di 1
- **zero (ZF)**: vale 1 se l'ultima operazione ha come risultato 0
- **sign (SF)**: vale 1 se l'ultima operazione ha prodotto un risultato negativo
- **overflow (OF)**: vale 1 se il risultato dell'ultima operazione supera la capacità di rappresentazione
- **interrupt enable (IF)**: indica se c'è la possibilità di interrompere l'esecuzione del programma in corso
- **direction (DF)**: modifica il comportamento delle operazioni su stringhe

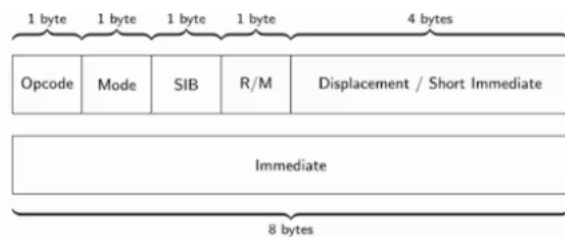
Sign lavora con il complemento a 2 . Gli ultimi due bit sono bit di controllo .

Andiamo a vedere come implementare le istruzioni nello z-64 :

Sono organizzate in otto classi:

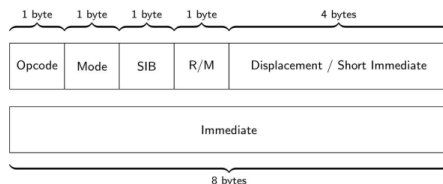
- Classe 0: Controllo dell'hardware
- Classe 1: Spostamento dati
- Classe 2: Aritmetiche (su interi) e logiche
- Classe 3: Rotazione e shift
- Classe 4: Operazioni sui bit di FLAGS
- Classe 5: Controllo del flusso d'esecuzione del programma
- Classe 6: Controllo condizionale del flusso d'esecuzione del programma
- Classe 7: Ingresso/uscita di dati

Mentre per quello che riguarda il formato delle istruzioni macchina :

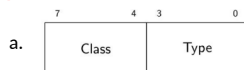


Il primo è un codice identificativo numerico che specifica una opportuna operazione (semantica istruzione), il secondo descrive il modo nel quale istruzione deve essere eseguita (vede opcode), il terzo serve a descrivere come vogliamo accedere in memoria, il quarto specifica se gli operandi sono registri di uso generale o qualche locazione di memoria, mentre l'ultimo si usa per gestire le costanti.

Torniamo al formato istruzioni macchina:

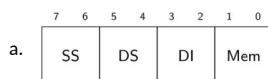


1. Op-code:



- b. Dove nella Class ci vanno i 4 bit che servono a specificare il tipo/famiglia delle istruzioni, mentre type specifica la precisa istruzione della famiglia.

2. Mode:



- b. SS rappresenta la codifica della dimensione dell'operando sorgente (2 bit).
 c. DS rappresenta la codifica della dimensione dell'operando destinazione (2 bit).
 d. DI (2 bit indipendenti) è un campo di 2 bit che indica o meno la presenza di un displacement o di un dato immediato.
 e. MEM indica quali degli operandi (sorgente / destinazione) sono da considerarsi operandi in memoria.

Campo	Valore	Significato
SS	00	La sorgente è un byte
	01	La sorgente è una word
	10	La sorgente è una longword
	11	La sorgente è una quadword
DS	00	La destinazione è un byte
	01	La destinazione è una word
	10	La destinazione è una longword
	11	La destinazione è una quadword
DI	00	Spiazzamento non utilizzato, immediato non presente
	01	Immediato presente
	10	Spiazzamento utilizzato
	11	Spiazzamento utilizzato, immediato presente
Mem	00	Sia la sorgente che la destinazione sono registri
	01	La sorgente è un registro, la destinazione è in memoria
	10	La sorgente è in memoria, la destinazione è un registro
	11	Condizione impossibile (genera un'eccezione a runtime)

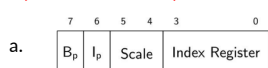
- g. Non esiste nessun tipo di configurazione che permette di fare una copia da memoria a memoria. Devo usare quindi un registro intermedio.
 h. Vediamo ora le modalità di indirizzamento dello Z-64: ricordiamoci che è architettura CISC, ma ci semplifica le cose (simile ad accesso a vettori): gli indirizzi non sono numeri di cella ma usa una formula:

i.

$$\left[\begin{matrix} AX \\ BX \\ CX \\ DX \\ SP \\ BP \\ SI \\ DI \\ R8 \\ R9 \\ R10 \\ R11 \\ R12 \\ R13 \\ R14 \\ R15 \end{matrix} \right] + \left[\begin{matrix} AX \\ BX \\ CX \\ DX \\ SP \\ BP \\ SI \\ DI \\ R8 \\ R9 \\ R10 \\ R11 \\ R12 \\ R13 \\ R14 \\ R15 \end{matrix} \right] * \begin{Bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{Bmatrix} + [\text{spiazzamento}]$$

- j. Partiamo dallo spiazzamento: 32 bit (calcolato a tempo scrittura / esecuzione), il quale viene sommato al resto, mentre per il registro di base (general purpose) viene usato come indirizzo base (esempio uso registro DX come locazione 0), infine per quanto lo spiazzamento (come muoverci a celle successive): in base al tipo di dato si ha la dimensione della cella opportuna (1,2,4,8 -> word, long-word, double-word, quad-word). Attenzione al contesto.

3. Sib (scale /index/base):



- b. I primi due bit indicano se la modalità di indirizzamento in memoria usa una base e/o indice
- c. Se $lp=1$ il campo index mantiene la codifica binaria del registro indice. 64 bit e solo scrittura.
- d. La scala invece vale 1,2,4,8 i quali vengono codificati con 2 bit

i.

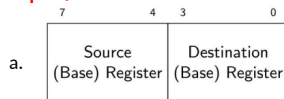
1	00
2	01
4	10
8	11

4. Codifica registri :

	Nome	Codifica	Uso Comune
	RAX	0000	Registro Accumulatore
	RCX	0001	Registro Contatore
	RDX	0010	Registro Dati
	RBX	0011	Registro Base
	RSP	0100	Stack Pointer
	RBP	0101	Base Pointer
	RSI	0110	Registro Sorgente
a.	RDI	0111	Registro Destinazione
	R8	1000	Registro di uso generale
	R9	1001	Registro di uso generale
	R10	1010	Registro di uso generale
	R11	1011	Registro di uso generale
	R12	1100	Registro di uso generale
	R13	1101	Registro di uso generale
	R14	1110	Registro di uso generale
	R15	1111	Registro di uso generale

- b. Da R8 a R15 possono essere usati per qualunque cosa
- c. RAX si usa come accumulatore , ci salvo le operazioni : registro usato come destinazione.
- d. RCX si usa come contatore : conta le istruzioni che verranno ciclitate .
- e. RDX : interazione con periferiche I/O.
- f. RBX: registro base : general purpose
- g. RSP : stack pointer
- h. RBP : base pointer
- i. RSI : sorgente : opera su stringa (buffer di memoria)
- j. RDI : destinazione : opera su stringa (buffer di memoria)

5. Campo R/M :

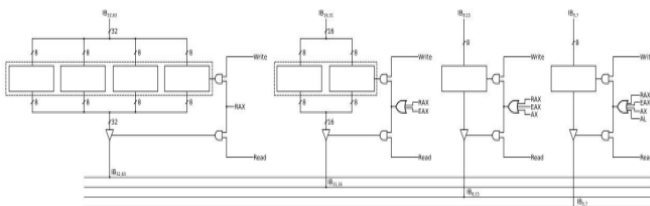


- b. Sorgente sempre RAX (0000)
- c. Entrambi i campi sono codificati due registri
- d. L'interpretazione di questi due campi dipende sempre dal contesto : dipende dalla configurazione di mem e da quella di Bp
- e. Possibile quindi interpretare registri come semplici registri oppure come registri base.

Riassumendo l'indirizzamento nel processore : per l'indirizzo il processore esegue e soddisfa l'equazione :

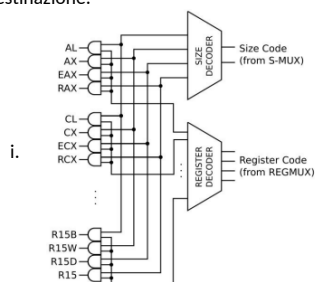
Disp(Rbase (64 bit), Rindice (64 bit), scala (tipi primitivi 8,16,32,64) , spiazzamento 32 bit)

Vediamo ora come funzionano i registri virtuali (scrivo e leggo solo i ff che mi servono) senza usarli tutti :



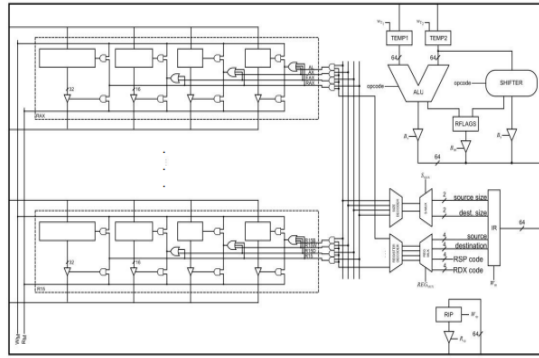
In dettaglio :

1. Si parte dai più significativi a sx verso i meno significativo a dx.
2. Si propaga un segnale W/R se e solo serve a tutti (OR a tutti i registri) .
3. I buffer-tri-state servono a fare da interruttore , ed in base alla porzione che devo usare del registro, e con una AND decide se scrivere o no .
4. Quindi porte AND e OR abilitano la lettura/scrittura nei vari blocchi (blocchi da 8 bit-> 1 byte) : specificano la porzione virtuale di quello fisico .
5. Così aggiungo 4 segnali in più in input alla rete sequenziale .
6. Per ovviare a ciò , vado a vedere il campo MODE : data la codifica usando un Decoder , genero in uscita una sola uscita che serve per generare uno dei quattro segnali di controllo W/R , andando così a scrivere nella parte corretta del registro virtualizzato (8,16,32,64) sia della sorgente , sia della destinazione:



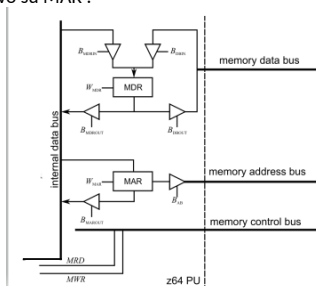
- ii. Gli input di questi decoder li prendo da due Mux : prendono la codifica di R/M e SS , mentre per la destinazione R/M e DS , arrivando al seguente circuito :

1.



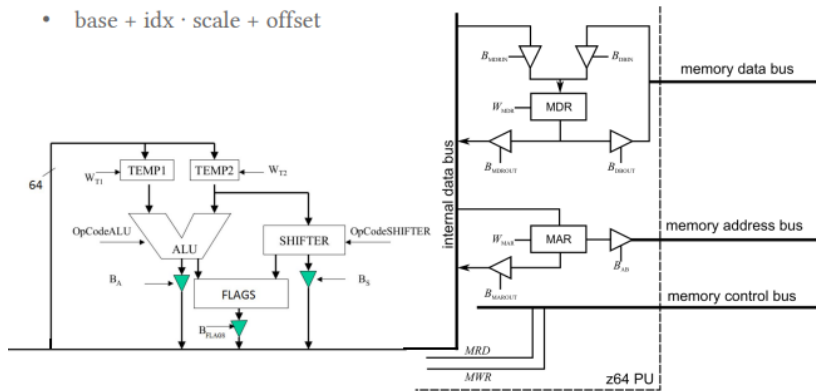
2. Così dall'Alu a posto di 6 bit di controllo, ne usiamo 1 (sia per sorgente e destinazione) : ma in realtà non basta : Ingressi aggiuntivi da IR . Può capitare che IR non sappia tutte le info, in quanto alcune istruzioni (costanti) siano già memorizzate in registri appositi . Come le generiamo le costanti ? Uso il registro FLAGS , il quale al suo interno c'è un valore 1 (posizione 1), il quale può essere collegato in vari modi per ottenere la costante.

Tornando alla modalità di indirizzamento : prima la moltiplicazione e poi 2 addizioni : si potrebbe usare gli adder per fare queste operazioni, ma aumenta la complessità e la circuiteria quindi optiamo per lo Shifter : il risultato della moltiplicazione viene salvato dove? In un registro general purpose? Assolutamente no : (sovrascrivo dati con altri dati temporanei) , **quindi uso un registro tampone** (invisibile al programmatore) , ma facendo così aggiungo 64 FF .. Nel nostro caso si usa il registro MAR , che viene usato come registro accumulatore : indice * scala , poi lo passo ad alu -> indice*scala + base , e poi lo riscrivo su MAR :



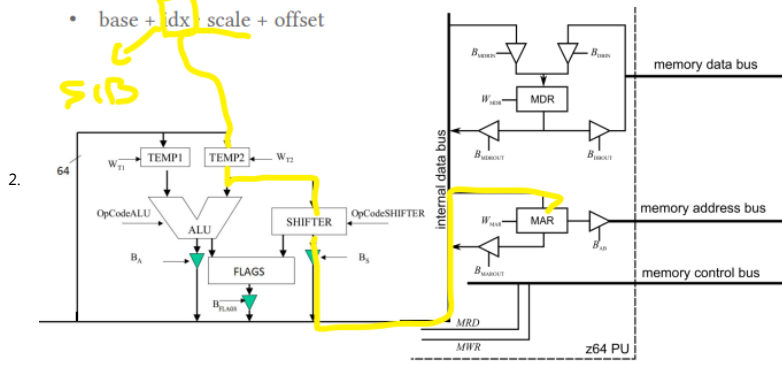
Possiamo usare questo circuito ? Assolutamente no , perché il flusso dati su data bus interno è unidirezionale . Quindi devo aggiungere un collegamento diretto tra MAR e il resto della circuiteria , arrivando a questo circuito :

- $base + idx \cdot scale + offset$

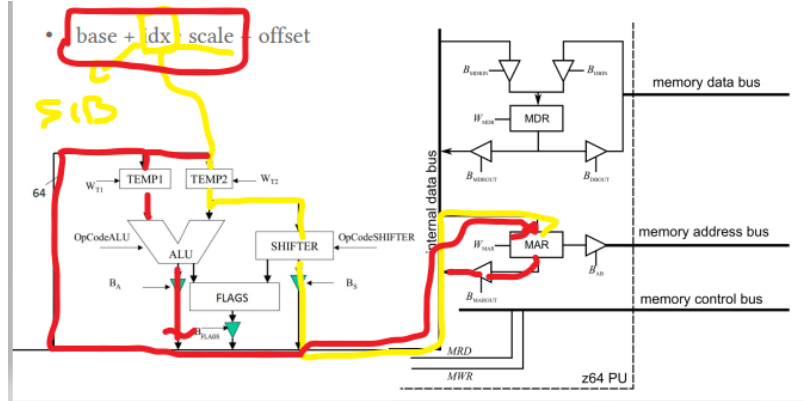


1. Il contenuto del Registro index (sotto porzione di SIB) viene fatto passare sul data bus interno , fino ad arrivare a Temp2. Poi lo mando allo shifter il quale ha come op-code la quantità da shiftare (1,2,4,8) , abilito uscita dello shifter , viaggia su internal data bus, arriva a MAR , ed abilito scrittura : su MAR ho indice*scala:

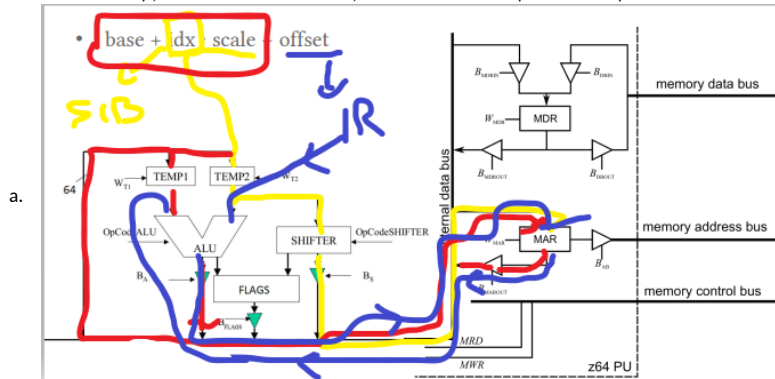
- $base + idx \cdot scale + offset$



3. Dopo aver scritto su MAR, abilito la scrittura, quindi buffer-tri-state, dati viaggiano sul bus interno dati ed arrivano fino a Temp1 (indice*scala), mentre su Temp2 c'è l'indirizzo base, poi vado nell'alu che viene settata per fare la somma, abilito buffer-tri-state e lo faccio viaggiare sul bus interno dati, lo scrivo su MAR, abilitando il segnale di scrittura, avendo così base+(indice*scala):



5. Vediamo ora per lo spiazzamento: 4 byte meno significativi della codifica istruzione (32 bit meno significativi) da IR: me li salvo in un registro temporaneo, poi abilito la lettura da MAR, lo salvo in altro registro temporaneo, configuro alu per fare somma, abilito buffer-tri-state (dati viaggiano su bus dati interno), abilito scrittura su MAR, avendo così base + (indice*scala) + offset:



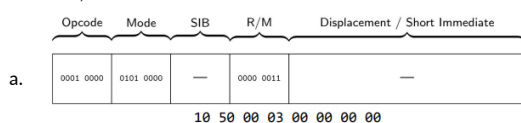
Questo processo è possibile visto che il processore è multi-ciclo: ogni istruzione in un colpo di clock differente: **ogni micro-istruzione viene eseguita ogni colpo di clock**: scrivi index dentro temp2, moltiplica usando shifter e scrivi dentro MAR, terza copia MAR dentro un registro temporaneo.

Andiamo ora a vedere alcune istruzioni assembly:

- Movb %al,%bl
 - Spostamento dati tra registri virtuali. Istruzione solo sugli 8 bit meno significativi
- Movw %ax,(%rdi)
 - %rdi è operando in memoria. Quindi muovo in %ax l'indirizzo base in cui vado a fare scrittura di due byte.
- Movl (%rsi), %eax
 - Dest reg virtuale a 32 bit, src a 64 bit. Quindi vado ad indirizzo puntato da %rsi, prendo 4 byte (long-word), li porto nel processore e poi li scrivo nella virtualizzazione a 32 di RAX (nel nostro caso virtualizzato a 32 -> %eax)
- Movq (%rsi,%rcx,8), %rax
 - %rsi è la base, %rcx indice, 8 scala
 - Copia in %rax il contenuto degli 8 byte della memoria il cui indirizzo iniziale è calcolato come $RSI+RCX*8$
- Subl %eax,%edx
 - Sottrazione tra due registri a 32 bit.
 - Faccio b-a
 - Lo scrivo nel registro destinazione.
 - Usa il complemento a 2 per fare la somma
- Add \$d,%al
 - Da notare che "i"\$" sta a simboleggiare una costante, altrimenti prendo il byte in memoria alla cella "d"

Andiamo a vedere ora in dettaglio come funzionano le funzioni (con tutti i campi). **Ogni bit in hex rappresenta la codifica dei 4 bit di ogni segnale:**

- Movw %ax,%bx



- Dove la codifica in esadecimale rappresenta i vari segnali di controllo espressi nell'istruzione
- In questo tipo di indirizzamento non si ha nessun accesso in memoria, nessuna costante coinvolta (displacement tutto 0) e gli operandi sono a 2 byte (double word).

- Addb \$0xa,%al
 - Utilizza una costante numerica come operando
 - Non utilizza spiazzamento
 - Se prefisso "0x" si parla di costanti esadecimali

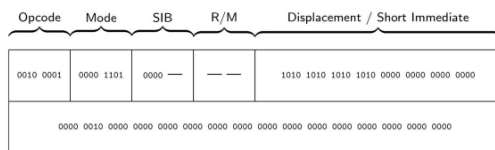


0010 0000	0000 0100	—	— 0010	0000 1010 0000 0000 0000 0000 0000
-----------	-----------	---	--------	------------------------------------

d. 20 04 00 02 0A 00 00 00

3. Subb \$0x2,0xAAAA

- Costante numerica come operando
- Presente uno spiazzamento



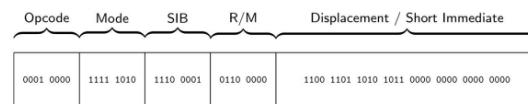
c.

21 0D 00 00 AA AA 00 00 02 00 00 00 00 00 00 00

- AAAA è indirizzo in memoria espresso in esadecimale, quindi va nello spiazzamento.

4. Movq 0xabcd(%rsi,%rcx,4),%rax

- Full addressing (base(%rsi), indice, scala e spiazzamento)



10 F8 E1 60 CD AB 00 00

5. Attenzione all'ordine dei bit nello spiazzamento : è invertito (da primo secondo) a (Secondo primo) : dovuto alla codifica che usiamo : little endian:



- Se lavoro con le long word (16 bit/ 2 byte) ok , ma se volessi lavorare con singole word(8 bit/1 byte) ?? Non possiamo lavorarci , ed in questo caso usiamo il **troncamento** : forzo da 16 a 8 , ma perdo il byte più significativo . Quindi il processore prende solo cella contenente "CD" e per trovare cella che contiene "AB" deve andare in avanti di n byte. Questo procedimento si ripete in base alla grandezza del dato:

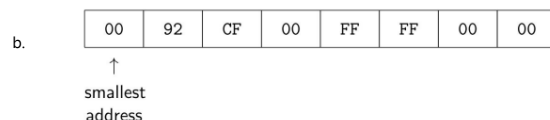


In generale : si parla di memory endianness : **ovvero di come le architetture codificano i dati** : si parla di **Big Endian** e di **Little Endian** : nel primo caso le codifiche dei dati avvengono da più significativo a quello meno significativo (vedendo lo stesso indirizzo di memoria) , mentre nella secondo si parte dal bit meno significativo fino a quello più significativo . I processori intel usando questa codifica . Vediamo un esempio :

Il valore 0xAABBCCDD è posto nel layout di memoria come segue:

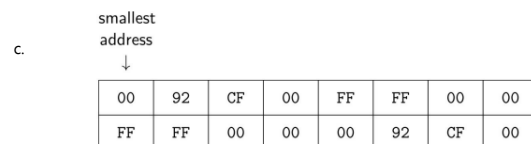


Prendiamo ad esempio 0x00cf9200 e 0x0000ffff



- Stesso ordine : vengono scambiate solo i byte , ma non l'ordine

Prendiamo ad esempio 0x00cf9200, 0x0000ffff e 0x00cf920000ffff



- Riassumendo il tutto : questa codifica ed in generale qualunque codifica (istruzioni , dati ecc ecc) è sconosciuta al processore , quindi il programmatore deve fornirla : La codifica delle istruzioni dipende dal contesto .**

Andiamo a vedere in dettaglio quali sono le istruzioni nello z64 (li suddividiamo in gruppi):

- B
 - Operando è un registro di uso generale , in indirizzo di memoria o un valore immediato.
- E
 - Operando è registro di uso generale o indirizzo di memoria.
- G
 - Operando di uso generale
- K
 - Operando è costante di 32 bit unsigned
- M
 - Operando è locazione di memoria , codificata come uno spiazzamento a partire da dal contenuto di RIP, dopo esecuzione fase fetch (punta a prossima istruzione):
 - In dettaglio:

Rip punta a questo indirizzo	Dopo fetch punta qui						
------------------------------	----------------------	--	--	--	--	--	--

- c. Ma cosa succede se il programma implementa un if ? (si parla di salti) : nel nostro caso si usa istruzione jump (salta ad un certo punto del programma : cambia indirizzo che non è sequenziale) . Con il jump la distanza in byte si calcola con lo spiazzamento e non con indirizzo completo.

6. K:

- a. Costante immediata a 32 bit .

Andiamo ora a vedere effettivamente le classi di operazioni dello z64 : ricordiamo che sono 8

1. Classe 0 : Controllo hardware

- a. Modificano lo stato della CPU , oppure eseguono istruzioni particolari

Tipo	Mnemonic	Operandi	O S Z P C	Descrizione
1	hlt	-	- - - -	Mette la CPU in modalità di basso consumo energetico, finché non viene ricevuta l'interruzione successiva
2	nop	-	- - - -	Nessuna operazione
3	int	Imm8	- - - -	Chiama esplicitamente un gestore di interruzioni

- c. HTL= arresta : **Disattiva il ciclo decode-fetch-execute.**
d. NOP = consuma byte: in exec non fa niente . Inserisce spazio in parti diverse del codice .
Serve per riallineare le istruzioni
e. INT= sincronizzazione per attivare gestore di dispositivi
f. **Richiamo di programma : STACK**
i. Struttura dati di tipo LIFO : Least in first out : inserisco e prelevo (ultimo) elemento .
ii. In dettaglio:

EI n-1
...
EI 2
EI 1
EI 0

- iii. Con push inserisco elemento, con pop prelevo ultimo elemento e lo invalido.
iv. Viene usata in quanto i registri sono limitati
v. In generale è composto da quad-word (64 bit -> 8 byte) : se di dimensione minore, il processore lo estende
vi. Per sapere quale è elemento affiorante della nostra pila : **Lo Stack Pointer** . Mantiene al suo interno indirizzo elemento affiorante. Non viene usato esplicitamente dal programmatore . Questo stack di programma viene allocato alla fine della memoria (elemento più grande), quindi per inserire nuovo elemento , lo devo decrementare.

2. Classe 1 : istruzioni di movimento dati

Tipo	Mnemonic	Operandi	O S Z P C	Descrizione
0	mov	B, E	- - - -	Fa una copia di B in E
1	movzX	E, G	- - - -	Fa una copia di E in G con estensione del segno
2	movzX	E, G	- - - -	Fa una copia di E in G con estensione dello zero
3	lea	E, G	- - - -	Valuta la modalità di indirizzamento, salva il risultato in G
4	push	E	- - - -	Copia il contenuto di E sulla cima dello stack
5	pop	E	- - - -	Copia il contenuto della cima dello stack in E
6	pushf	-	- - - -	Copia sulla cima dello stack il registro FLAGS
7	popf	-	- - - -	Copia nel registro FLAGS il contenuto della cima dello stack
8	movs	-	- - - -	Esegue una copia memoria-memoria
9	stos	-	- - - -	Imposta una regione di memoria ad un dato valore

b. Mov

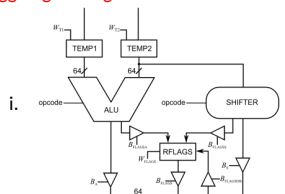
- i. Operando da B ad E : copia soltanto
ii. **Attenzione al contesto : esistono varie istruzioni per fare la stessa operazione**
iii. MOVsX :
1. Effettua la copia + estensione del segno (attenzione se aritmetica a CP2)
iv. MOVzX:

1. Estensione con lo "0" : interi positivi

Istruzione	Tipo di conversione
movsbw %al, %eax	Estendi il segno da byte a word
movsbl %al, %eax	Estendi il segno da byte a longword
movsbq %al, %rax	Estendi il segno da byte a quadword
movswl %ax, %eax	Estendi il segno da word a longword
movswq %ax, %rax	Estendi il segno da word a quadword
movslq %eax, %rax	Estendi il segno da longword a quadword

c. LEA (load effective address)

- i. Calcola indirizzo effettivo :
1. Movq disp(%rax,%rcx,8),%rdx
a. Così accedo a Base+(indice*scala)+offset
b. Prende il valore contenuto
2. Lea : valuta modalità indirizzamento e scrive nel registro il valore dell'indirizzo
a. Lea disp(%rax,%rcx,8),%rcx
b. Non accedo in memoria per sapere il contenuto in memoria .
d. Torniamo allo stack : PUSHF e POPF: permettono di effettuare una copia del registro flags (non modificabile dal programmatore) nello stack : **devo quindi modificare hardware :**
aggiungo collegamento da data bus interno e flags .



- e. MOVS (move string) : istruzione di tipo Stringa : copia memoria-memoria (dimensione maggiore di 8 byte)
- f. STOS (store string) : inizializza area di memoria ad un valore scelto dal programmatore
3. Classe 2 : istruzioni logico aritmetiche :

Tipo	Mnemonico	Operandi	O S Z P C	Descrizione
0	add	B, E	0 0 0 0 0	Memorizza in E il risultato di E + B
1	sub	B, E	0 0 0 0 0	Memorizza in E il risultato di E - B
2	adc	B, E	0 0 0 0 0	Memorizza in D il risultato di E + B + CF
a. 3	sbb	B, E	0 0 0 0 0	Memorizza in D il risultato di E - (B + neg(CF))
4	cmp	B, E	0 0 0 0 0	Confronta i valori di B ed E calcolando E - B, il risultato viene poi scartato
5	test	B, E	0 0 0 0 0	Calcola l'and logico bit a bit di B ed E, il risultato viene poi scartato
6	neg	E	0 0 0 0 0	Rimpiazza il valore di E con il suo complemento a 2

- b. Possibile interagire anche con il carry flag : ADC
- i. Consente di superare il limite imposto dal numero di bit dei registri
- c. SBB (subtract with borrow):
- i. Sottrazione a precisione arbitraria
- ```
movq $operand_1_high, %rax
movq $operand_1_low, %rbx
movq $operand_2_high, %rcx
movq $operand_2_low, %rdx
addq %rbx, %rdx
adcq %rax, %rcx
```
- ii.
- d. CMP (compare) : effettua di confronto tra dati : non serve inserire un comparatore, in quanto i processori effettuano il confronto come una sottrazione:
- i. A=B??
1. Il processore esegue B-A : se vale 0 sono uguali : uso lo zero-flags!!
- ii. A>B
1. Il processore esegue B-A<0 : se si minore
- iii. **Non salva il risultato nella destinazione .**
- e. Test
- i. A=0? -> a&a=0 . Si punta sul riuso del processore , usando stessa circuiteria per varie operazioni
- f. Neg : Cp2 del valore : se mancasse questa istruzione, non si potrebbe fare la sottrazione.

| Tipo | Mnemonico | Operandi | O S Z P C | Descrizione                                                |
|------|-----------|----------|-----------|------------------------------------------------------------|
| 7    | and       | B, E     | 0 0 0 0 0 | Memorizza in E il risultato dell'and bit a bit tra B ed E  |
| 8    | or        | B, E     | 0 0 0 0 0 | Memorizza in E il risultato dell'or bit a bit tra B ed E   |
| g. 9 | xor       | B, E     | 0 0 0 0 0 | Memorizza in E il risultato dello xor bit a bit tra B ed E |
| 10   | not       | E        | 0 0 0 0 0 | Rimpiazza il valore di E con il suo complemento a uno      |
| 11   | bt        | K, E     | - - - - 0 | Imposta CF al valore del K-simo bit di E (bit testing)     |

#### 4. Classe 3 : istruzioni di rotazione e shift

| Tipo | Mnemonico | Operandi | O S Z P C | Descrizione                           |
|------|-----------|----------|-----------|---------------------------------------|
| 0    | sal       | K, G     | 0 0 0 0 0 | Moltiplica per 2, K volte             |
| 1    | sal       | G        | 0 0 0 0 0 | Moltiplica per 2, RCX volte           |
| 0    | shl       | K, G     | 0 0 0 0 0 | Moltiplica per 2, K volte             |
| 1    | shl       | G        | 0 0 0 0 0 | Moltiplica per 2, RCX volte           |
| 2    | sar       | K, G     | 0 0 0 0 0 | Dividi (con segno) per 2, K volte     |
| 3    | sar       | G        | 0 0 0 0 0 | Dividi (con segno) per 2, RCX volte   |
| 4    | shr       | K, G     | 0 0 0 0 0 | Dividi (senza segno) per 2, K volte   |
| a. 5 | shr       | G        | 0 0 0 0 0 | Dividi (senza segno) per 2, RCX volte |
| 6    | rcl       | K, G     | 0 - - - 0 | Ruota a sinistra, K volte             |
| 7    | rcl       | G        | 0 - - - 0 | Ruota a sinistra, RCX volte           |
| 8    | rcr       | K, G     | 0 - - - 0 | Ruota a destra, K volte               |
| 9    | rcr       | G        | 0 - - - 0 | Ruota a destra, RCX volte             |
| 10   | rol       | K, G     | 0 - - - 0 | Ruota a sinistra, K volte             |
| 11   | rol       | G        | 0 - - - 0 | Ruota a sinistra, RCX volte           |
| 12   | ror       | K, G     | 0 - - - 0 | Ruota a destra, K volte               |
| 13   | ror       | G        | 0 - - - 0 | Ruota a destra, RCX volte             |

- b. Attenzione al tipo : identifica due istruzioni diverse :
- i. Se a sx non c'è differenza (aritmetico = logico) :
1. 0011 | 1110 -> in entrambi mi darà 0110 | 1100 : faccio entrare il bit 0 a sx
- ii. Se a dx :
1. Attenzione al segno (aritmetico != logico) :
- a. 1000 | 0011 -> 1100 | 0001
- iii. Nel caso di shift , per stabilire la posizione di quanto shiftare ci sono due modi : o lo passo come secondo parametro oppure uso il registro contatore

#### 5. Classe 4 : Manipolazione di bit di flags

| Tipo | Mnemonico | Operandi | O S Z P C | Descrizione |
|------|-----------|----------|-----------|-------------|
| 0    | c1c       | -        | - - - - 0 | Resetta CF  |
| 1    | c1p†      | -        | - - - - 0 | Resetta PF  |
| 2    | c1z†      | -        | - - - - 0 | Resetta ZF  |
| 3    | c1s†      | -        | - - - - 0 | Resetta SF  |
| 4    | c1i       | -        | - - - - - | Resetta IF  |
| 5    | c1d       | -        | - - - - - | Resetta DF  |
| 6    | c1o†      | -        | 0 - - - - | Resetta OF  |
| a. 7 | stc       | -        | - - - - 1 | Imposta CF  |
| 8    | stp†      | -        | - - - - 1 | Imposta PF  |
| 9    | stz†      | -        | - - - - 1 | Imposta ZF  |
| 10   | sts†      | -        | - - - - 1 | Imposta SF  |
| 11   | sti       | -        | - - - - - | Imposta IF  |
| 12   | std       | -        | - - - - - | Imposta DF  |
| 13   | sto†      | -        | 1 - - - - | Imposta OF  |

†: non esiste un'istruzione corrispondente nell'assembly x86

- b. Cl = clear
- c. No move possibile , ma così si

6. Classe 5 : istruzioni di controllo del flusso del programma

| Tipo | Mnemonico | Operandi | O S Z P C | Descrizione                               |
|------|-----------|----------|-----------|-------------------------------------------|
| 0    | jmp       | M        | - - - -   | Esegue un salto relativo                  |
| 1    | jmp       | *G       | - - - -   | Esegui un salto assoluto                  |
| 2    | call      | M        | - - - -   | Esegue una chiamata a subroutine relativa |
| 3    | call      | *G       | - - - -   | Esegue una chiamata a subroutine assoluta |
| 4    | ret       | -        | - - - -   | Ritorna da una subroutine                 |
| 5    | iret      | -        | ⇕ ⇕ ⇕ ⇕   | Ritorna dal gestore di una interruzione   |

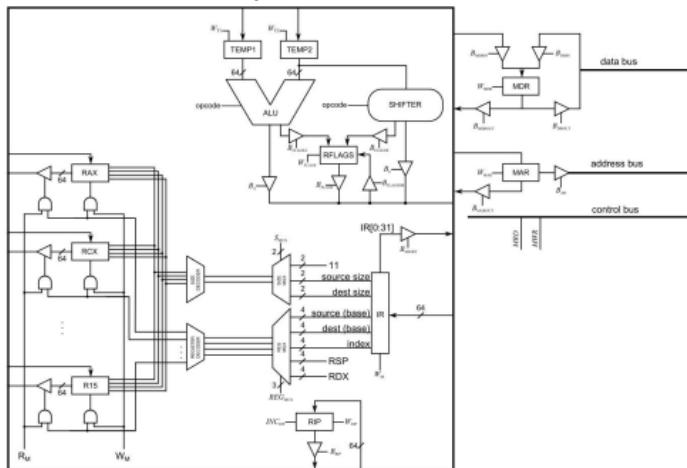
- a.
- b. Di solito esecuzione istruzione è sequenziale , ma possiamo alterare questo flusso : JMP e CALL . Nel primo si salta ad un indirizzo distante : il processore salta a punto esplicito ( M=offset) : quindi il processore aggiorna RIP , non tenendo conto di indirizzo partenza ; nel secondo caso chiamo le sub-routine ( il flusso di controllo viene trasferito a prima istruzione della sub-routine ) . Il ritorno invece si ha ad istruzione dopo invocazione sub-routine (effettuata con istruzione ret); Iret variante che conclude interazione / esecuzione driver. **Gli argomenti di jump e call possono essere indirizzo e/o registro . Spiazzamento di RIP (salto relativo) / indirizzo a 64 bit nel registro (salto assoluto) .**

7. Classe 6 : Controllo condizionale del flusso

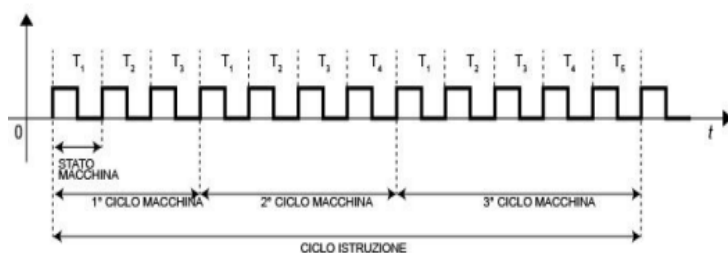
| Tipo | Mnemonico | Operandi | O S Z P C | Descrizione                     |
|------|-----------|----------|-----------|---------------------------------|
| 0    | jc        | M        | - - - -   | Salta a M se CF è impostato     |
| 1    | jp        | M        | - - - -   | Salta a M se PF è impostato     |
| 2    | jz        | M        | - - - -   | Salta a M se ZF è impostato     |
| 3    | js        | M        | - - - -   | Salta a M se SF è impostato     |
| 4    | jo        | M        | - - - -   | Salta a M se OF è impostato     |
| 5    | jnc       | M        | - - - -   | Salta a M se CF non è impostato |
| 6    | jnp       | M        | - - - -   | Salta a M se PF non è impostato |
| 7    | jnz       | M        | - - - -   | Salta a M se ZF non è impostato |
| 8    | jns       | M        | - - - -   | Salta a M se SF non è impostato |
| 9    | jno       | M        | - - - -   | Salta a M se OF non è impostato |

- b. Se JNZ faccio salti se condizioni sono 0.

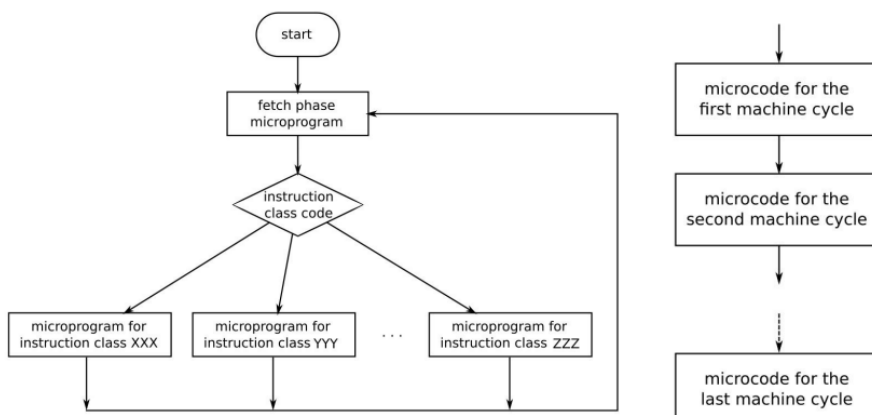
Quindi l'architettura finale dello z-64 è la seguente :



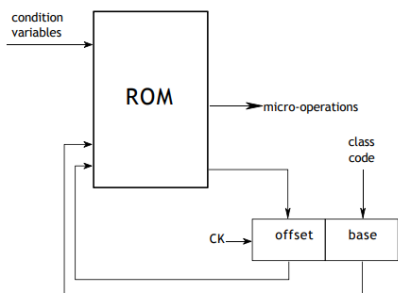
**Andiamo ora a vedere l' unità di controllo :** l'unità di controllo implementa le micro-operazioni : quindi unità di controllo è un'insieme di micro-programmi :



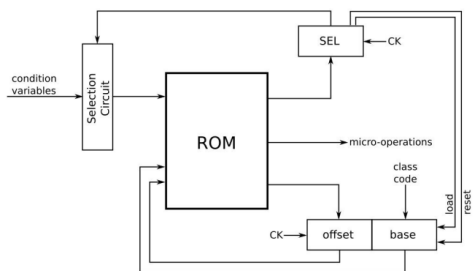
Quindi la CU è una grande macchina a stati finiti , la quale esegue micro-operazioni in più cicli macchina (visto che è multi ciclo) .In dettaglio :



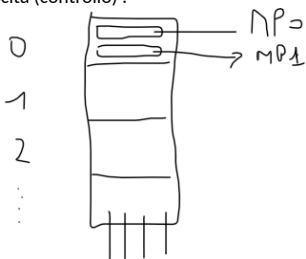
In dettaglio: gli input alla mia unità di controllo ha diversi input: IR, registro Flags (operazione sì / no), eventuali segnali da dispositivi esterni (uso data bus): quindi gli input dipende da come abbiamo implementato l'automa, mentre per quanto riguarda gli output dipende da come è stata implementata l'unità di processing. In dettaglio:



Ma questo schema ha un problema : quanti segnali in input ha ?? Andiamo a vederlo :



In questo circuito per circuito di selezione si intendono un numero di mux , il cui output è il massimo numero di variabili che vengono processate. Inoltre la rom fornisce i segnali di controllo per il selection. L'offset invece viene implementato come incremento, vista la sequenzialità delle micro-istruzioni. Non sempre rispetta questa meccanica, infatti se si parla di salti (esempio jz) , l'offset non è sequenziale . Ricordiamoci della Rom Paginata(già vista): la forma della Rom da rettangolare a quadrata: in ogni pagina ci vanno le micro istruzioni e per ogni riga di ogni pagina, ci sono le codifiche (circuitali) dei segnali di uscita (controllo) :

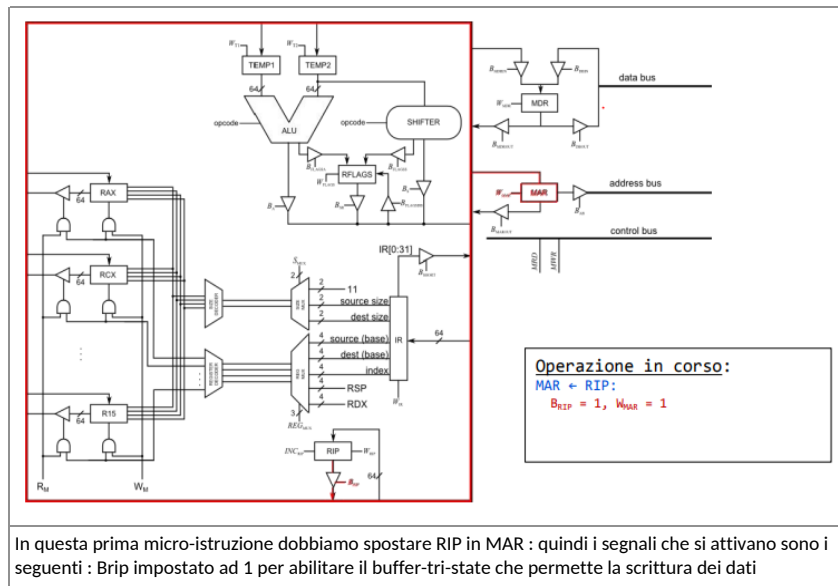


Dove a sx si intendono il numero delle pagine della Rom , mentre a destra si intendono le micro-istruzione del micro-programma contenuto nella pagine . Infine sotto ci sono i segnali di uscita , derivanti dalla realizzazione circuitali della ROM. Attenzione alla frammentazione interna : micro-istruzioni occupano dimensione minore di quella allocata alla pagina della ROM.

Andiamo a vedere ora in dettaglio le micro-istruzioni : **iniziamo dal fetch** (micro-programma anche esso)

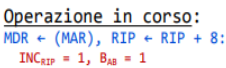
```
MAR ← RIP
MDR ← (MAR); RIP ← RIP + 8
IR ← MDR
```

In dettaglio :

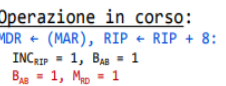


In questa prima micro-istruzione dobbiamo spostare RIP in MAR : quindi i segnali che si attivano sono i seguenti : Brip impostato ad 1 per abilitare il buffer-tri-state che permette la scrittura dei dati

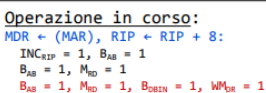
Seconda micro-istruzione:



Vediamo il secondo step :

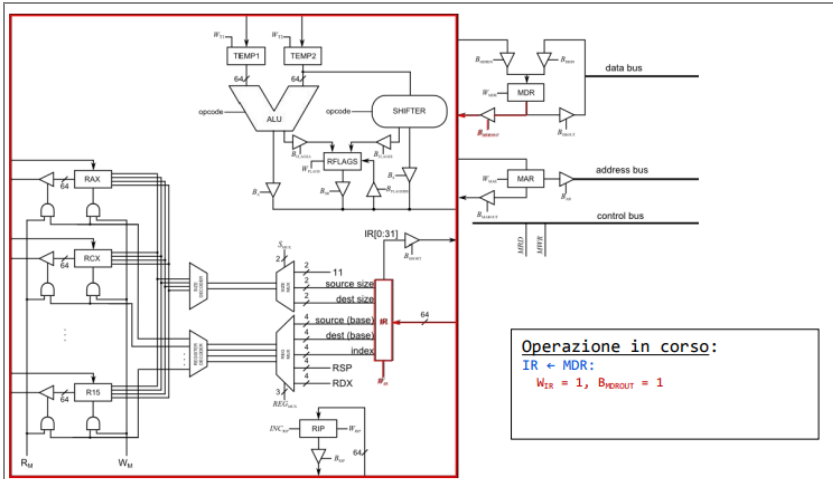


Ultimo colpo di clock : bufferizzo all'interno del processore i dati che stanno arrivando dal data bus



Per effettuare la scrittura finale su MDR, si mantiene il buffer-tri-state di MAR attivo (Bab=1), così i dati escono su address bus, si mantiene il control bus in scrittura (Mrd=1), si abilita il buffer-tri-state di MDR per fare entrare i dati dal data bus (Bdbin=1) e si abilita il segnale di scrittura su MDR (Wmdr=1)

Mentre nell'ultima fase della micro-istruzione andiamo a copiare MDR( codifica dell'istruzione) nell'IR :



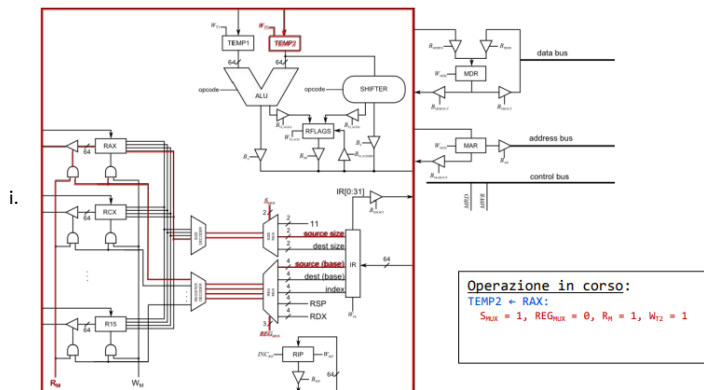
Visto che devo copiare il contenuto di MDR in IR, abilito il buffer-tri-state per permettere scrittura sull'internal bus data (Bmdrout=1), poi i dati viaggiano su internal bus data, fino ad arrivare all'IR: serve il segnale di scrittura abilitato per fare ciò:  $W_{IR}=1$ . **Quindi per fare la fase di fetch servono necessariamente 4 cicli di clock.**

Vediamo altri esempi :

`movq %rax, %rcx:`

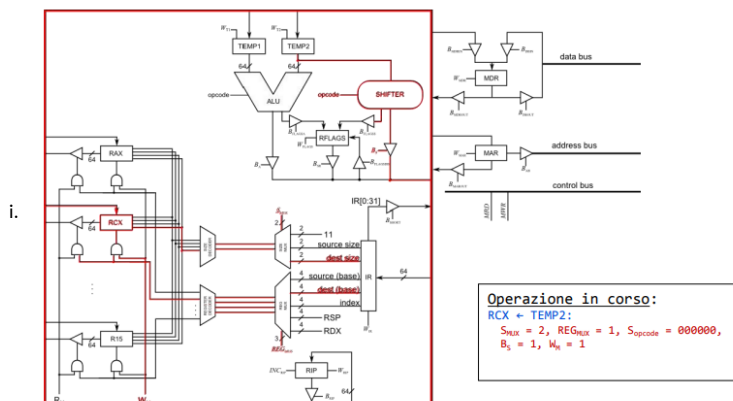
- $MAR \leftarrow RIP$
- $MDR \leftarrow (MAR); RIP \leftarrow RIP + 8$
- 1. •  $IR \leftarrow MDR$
- $TEMP2 \leftarrow RAX$
- $RCX \leftarrow TEMP2$

- a. Da notare che per qualunque micro-istruzione esegue la fase di fetch .
- b. Vediamo ora la micro-istruzione  $TEMP2 \leftarrow RAX$



- ii. Notiamo che usiamo TEMP2 (registro tampone) , non passo per la ALU, ma passo per lo shifter , il quale in questo caso fa lo shift di 0 posizioni : il dato passa così' come è ed inoltre perché posso usare 1 solo registro alla volta. Per attivare la lettura da RAX devo abilitare  $R_m = 1$  (da uc) , il quale permette di selezionare un qualunque registro, quindi per selezionare solo quel registro devo prendere gli input dei decoder : codice del registro e taglia del registro virtuale . Queste informazioni vengono prese dall'IR : codice sorgente e taglia sorgente : pilota i mux :  $S_{MUX}=1$  (taglia) e  $REG_{MUX}=0$  (sorgente), attivo buffer-tri-state di RAX (permette il viaggio dei dati su internal data bus) e raggiungo TEMP2. Infine per attivare la scrittura su TEMP2 , devo abilitare il segnale di write ( $W_{T2}=1$ ).

- c. Vediamo ora  $RCX \leftarrow TEMP2$

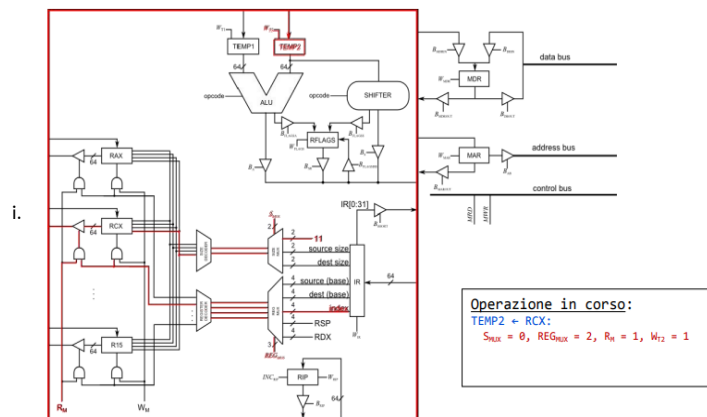




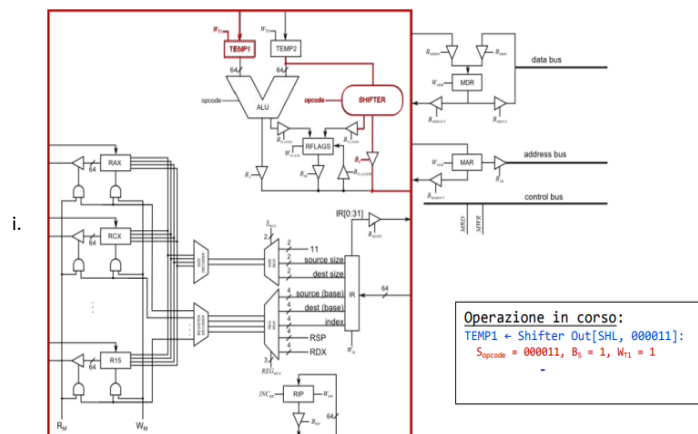
```
movq %rax, 0xaaaa(%rax, %rcx, 8):
```

- MAR  $\leftarrow$  RIP
- MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- IR  $\leftarrow$  MDR
- TEMP2  $\leftarrow$  RCX
- TEMP1  $\leftarrow$  Shifter Out[SHL, 000011]
- TEMP2  $\leftarrow$  RAX
- MAR  $\leftarrow$  ALU OUT[ADD]
- TEMP1  $\leftarrow$  IR[0:31]
- TEMP2  $\leftarrow$  MAR
- MAR  $\leftarrow$  ALU OUT[ADD]
- MDR  $\leftarrow$  RAX
- (MAR)  $\leftarrow$  MDR

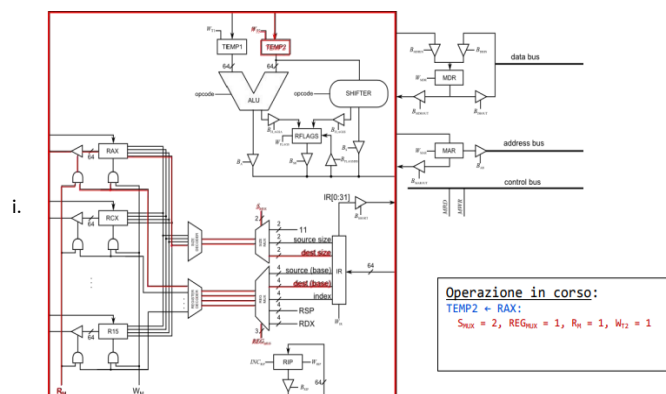
- Modalità di indirizzamento completa (base , indice , scala e spiazzamento)
- Partiamo da `TEMP2<-RCX`



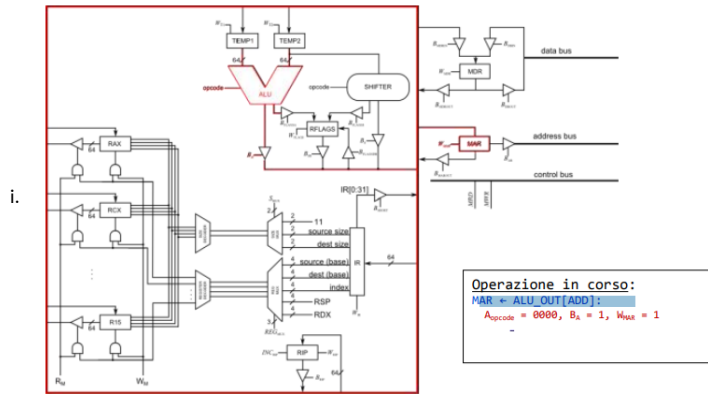
- c. TEMP1<-shifter out [shl,000011] : 000011 rappresenta il codice dello shifter (3) il che equivale a shiftare di 8 :



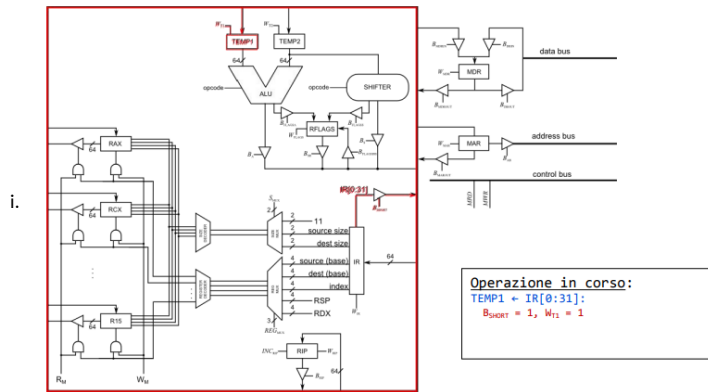
- ii. Dopo aver shiftato di 3 posizioni, abilito il buffer-tri-state per mandare i dati su internal data bus, e questi dati li scrivo su TEMP1, abilitando il segnale opportuno (Wtempo1=1) .
- d. TEMP2<-RAX:



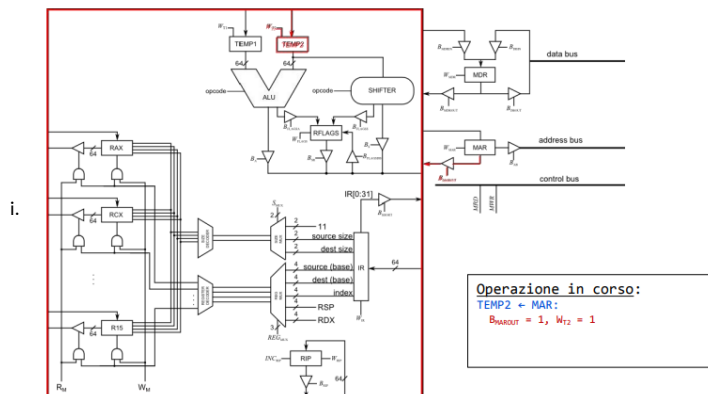
- ii. Setto i segnali nel giusto ordine : Smux=2 (taglia della destinazione), Regmux=1 (tipo del registro destinazione) , poi abilito il segnale di scrittura nei registri (Rm=1) ed infine abilito la scrittura su TEMP2 (Wt2=1)
- e.  $MAR \leftarrow ALU\_OUT[ADD]$ :



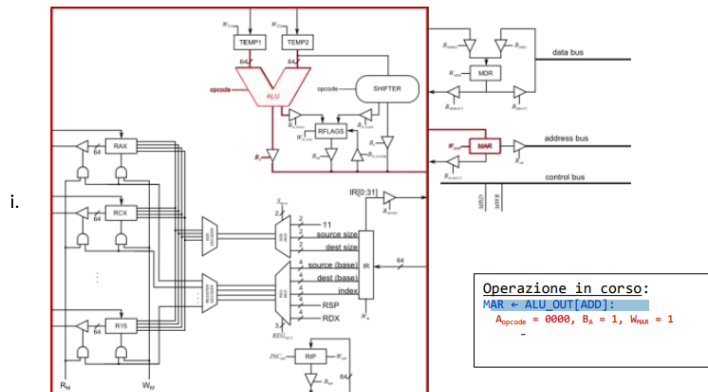
- ii. Qui passo per ALU: effettua la somma (Aopcode=0000) , abilito il buffer-tri-state per scrivere sull'internal data bus (Ba=1) ed infine scrivo su MAR , abilitando anche qui il segnale di scrittura (Wmar=1).
- f.  $TEMP1 \leftarrow IR[0:31]$  : cosi calcolo lo spiazzamento



- ii. Prendo i 32 bit dello spiazzamento da IR, abilito il suo buffer-tri-state (Bshort=1), cosi i dati viaggiano sull'internal bus data ed infine li scrivo in TEMP1, abilitando il segnale di scrittura (Wt1=1).
- g.  $TEMP2 \leftarrow MAR$ :

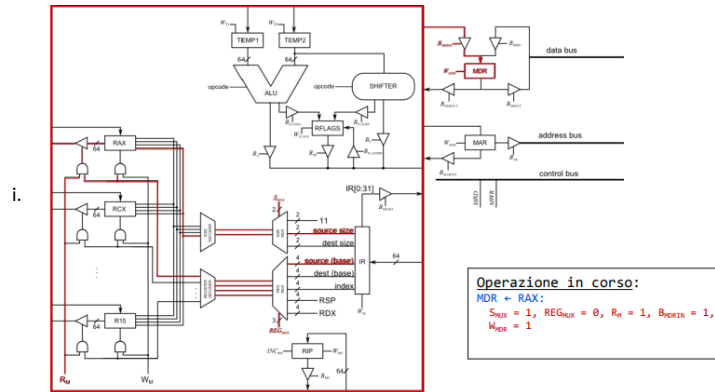


- ii. Qui abilito il buffer-tri-state di MAR per scrivere su interna bus data (Bmarout=1), cosi i dati viaggiano sull'internal bus data, arrivando a TEMP2 e vi vengono scritti in quanto il segnale di scrittura è abilitato (Wt2=1).
- h.  $MAR \leftarrow ALU\_OUT[ADD]$ :



- ii. Sposto base\*indice+ spiazzamento in MAR: faccio fare all'alu la somma (Aopcode=0000) , abilito il buffer-tri-state per scrivere su internal bus data (Ba=1) , facendo viaggiare i dati fino a MAR ed abilitando il segnale di scrittura (Wmar=1).

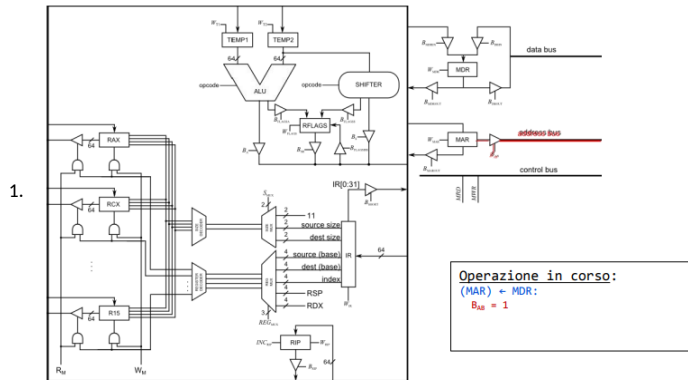
i. MDR<-RAX:



- ii. Cominciamo dai mux: Smux=1 in quanto devo lavorare con il registro sorgente (RAX) , RegMUX=0 (registro base) , poi abilito la lettura dai registri (Rm=1), abilito il buffer-tri-state per mandare i dati sull'internal data bus , poi li mando a MDR abilitando il buffer-tri-state in entrata (Bmdrin=1), ed infine scrivo il valore abilitando il segnale di scrittura (Wmdr=1).

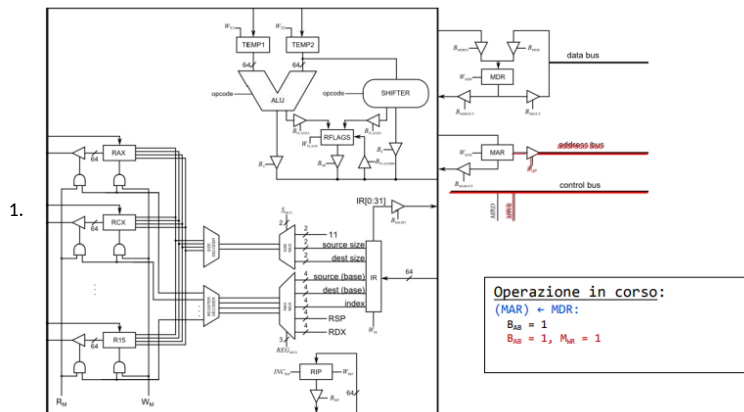
j. (MAR)<-MDR:

i. Primo step



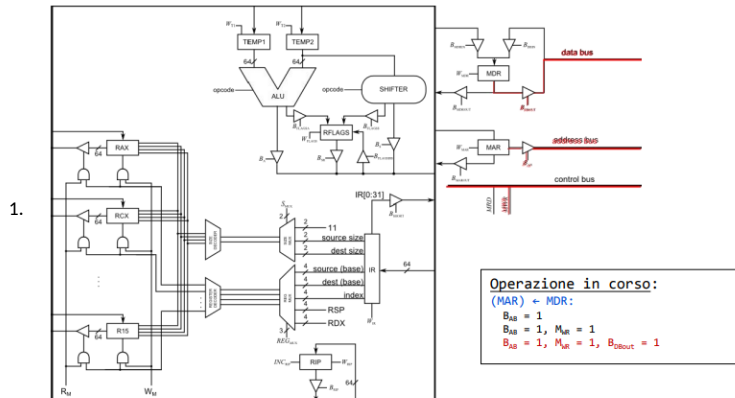
2. Abilito il buffer-tri-state di MAR per scrivere i dati sull'address bus (Bab=1)

ii. Secondo step :



2. Per seconda micro-istruzione , abilito il control bus, ed in particolare la scrittura su ste stesso : (Bab=1 e MWR=1)

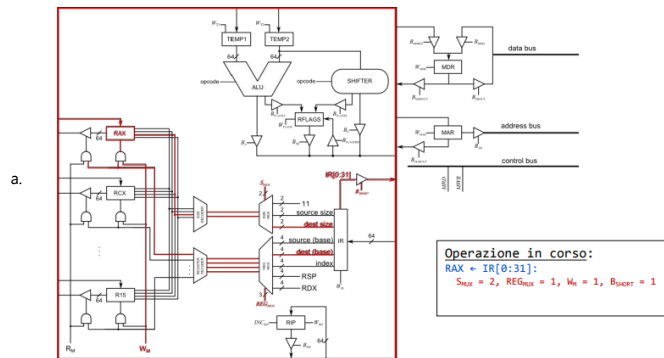
iii. Terzo step:



2. Abilito il buffer-tri-state di MDR (Bmdrout=1) così i dati viaggiano nel data bus.

```
movl $0xaaaa, %eax:
```

- MAR  $\leftarrow$  RIP
- 3. • MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- IR  $\leftarrow$  MDR
- EAX  $\leftarrow$  IR[0:31]



- b. Non c'è spiazzamento, e copia i 32 bit meno significativi di IR in RAX : seleziono Smux=2, Regmux=1, abilito il segnale per scrivere sui registri (Wm=1) e abilito anche il buffer-tri-state di IR (Bshort=1).

```
movq $0xaaaa, %rax:
```

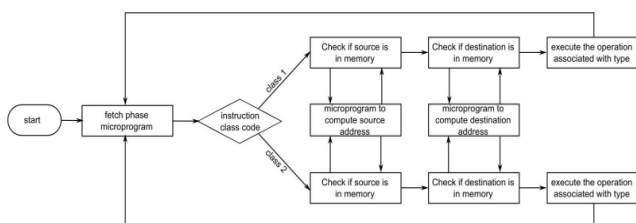
- MAR  $\leftarrow$  RIP
- MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- 4. • IR  $\leftarrow$  MDR
- MAR  $\leftarrow$  RIP
- MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- RAX  $\leftarrow$  MDR

- a. Qui stessa costante di prima, ma a 64 bit, quindi abbiamo bisogno di 128 bit.
- b. In questo caso IR punta nel mezzo dell'istruzione !!
- c. Devo per forza trasferire alla memoria : devo sistemare anche il IR.

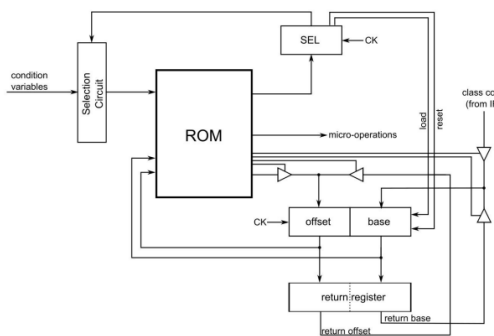
Quindi in base al tipo di operandi che si hanno, l'unità di controllo ha interazioni diverse (modalità di indirizzamento) : si cerca di ridurre il peso delle micro-operazioni, magari usando delle micro-funzioni/micro-programma (salvato nella ROM) :

```
1 if D == 1 and Bp == 0 and Ip == 0
2 MAR ← IR[0:31]
3 else if D == 0 and Bp == 1 and Ip == 0
4 MAR ← B
5 else if Ip == 1
6 TEMP2 ← I
7 MAR ← SHIFTER.OUT[SHL, T]
8 TEMP1 ← MAR
9 if D == 1
10 TEMP2 ← IR[0:31]
11 MAR ← ALU.OUT[ADD]
12 TEMP1 ← MAR
13 endif
14 if Bp == 1
15 TEMP2 ← B
16 MAR ← ALU.OUT[ADD]
17 endif
18 endif
```

I microprogrammi vanno a finire nelle pagine vuote della ROM ( ve ne sono 8 "Libere"). Quindi di conseguenza devo modificare la mia macchina a stati, che diventa :



Mentre il circuito diventa così :

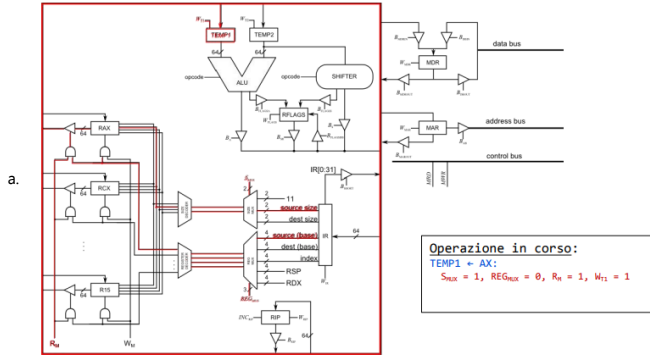


Vediamo ora altri esempi :

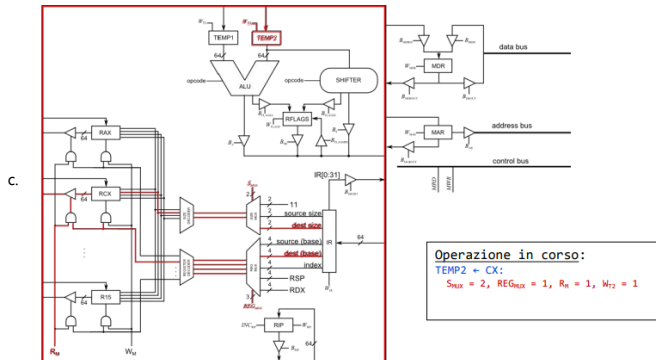
```
addw %ax, %cx:
```

- MAR  $\leftarrow$  RIP
- MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- 1. • IR  $\leftarrow$  MDR

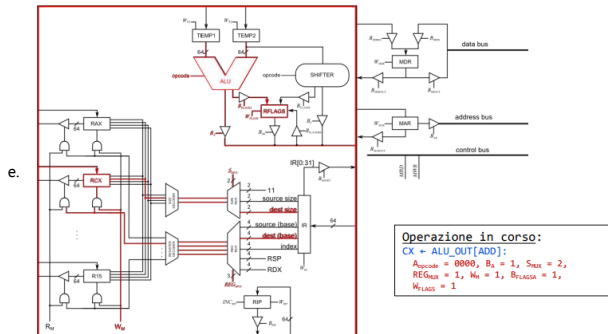
- TEMP1  $\leftarrow$  AX
- TEMP2  $\leftarrow$  CX
- CX  $\leftarrow$  ALU OUT[ADD]



- b. Cominciamo con il configurare il mux delle sorgenti a 1 (Sreg=1), quello delle taglie a 0 (Regmux=0), abilitiamo il segnale di lettura del banco registri (Rm=1), abilitiamo il buffer-tri-state del registro, così i dati viaggiano sull'internal bus data, arrivando a TEMP1 ed abilitiamo il segnale di scrittura (Wtemp1=1).



- d. visto che in questa micro-istruzione dobbiamo lavorare con un registro, settiamo i mux nei modi corretti : il primo (tipo) lo setto a dest(Smux=1), quello della taglia (secondo) lo setto ad 1 (Regmux=1)->registro base di dest, abilito il buffer-tri-state e lo mando a TEMP2, abilitando il segnale di scrittura su TEMP2(Wtemp2=1)



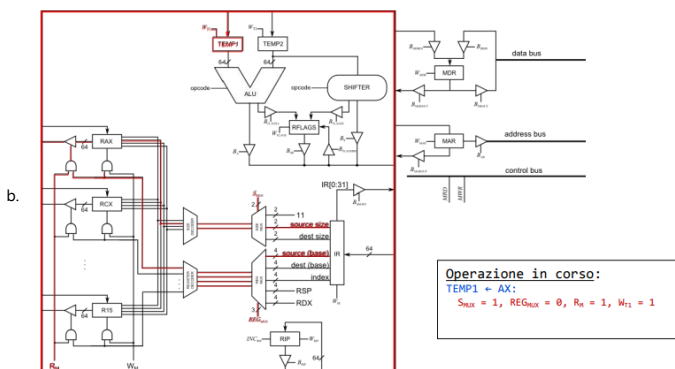
- f. Per quanto riguarda il settaggio dei mux è lo stesso di prima ; concentriamoci ora sull'ALU: gli si passa opcode 0000 , si abilita il buffer-tri-state che permette di scrivere sull'internal bus data (Ba=1), abilito il buffer-tri-state per scrivere nel registro flags(Bflagsa=1) ed abilito il segnale di scrittura su Rflags (Wrflags=1); infine per scrivere nel registro devo abilitare il segnale di scrittura nei registri (Wreg=1).

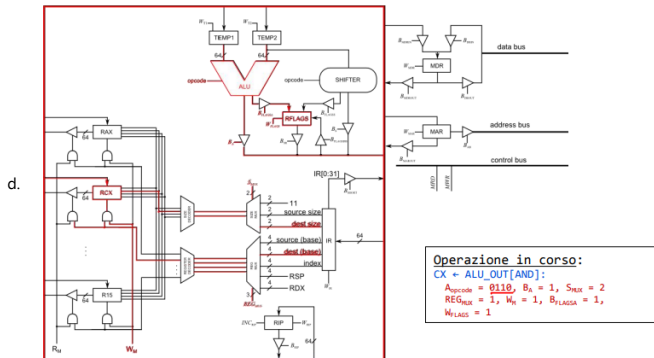
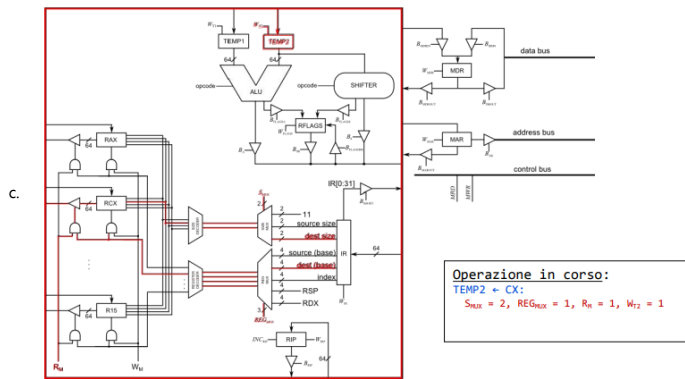
```
andw %ax, %cx:
```

- MAR  $\leftarrow$  RIP
- MDR  $\leftarrow$  (MAR); RIP  $\leftarrow$  RIP + 8
- IR  $\leftarrow$  MDR
- TEMP1  $\leftarrow$  AX
- TEMP2  $\leftarrow$  CX
- CX  $\leftarrow$  ALU OUT[AND]

2.

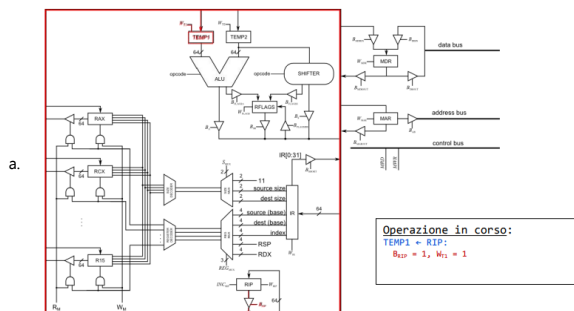
- a. Simile a quello di sopra , ma cambia l'opcode della ALU : da 0000 a 0110 :



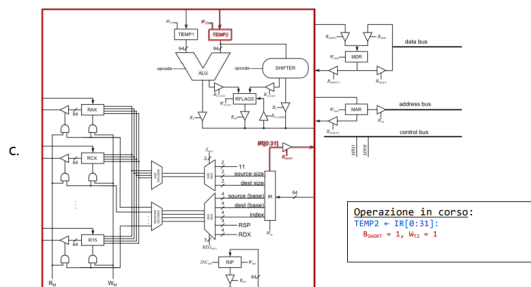


jz displacement:

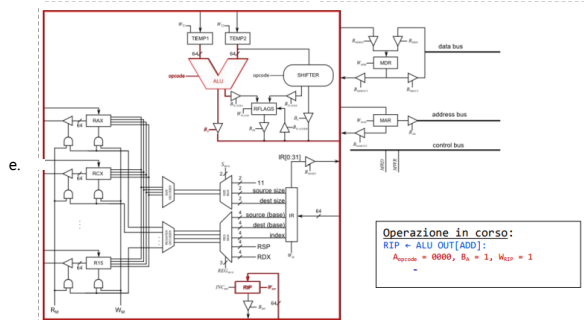
- \*  $MAR \leftarrow RIP$
- \*  $MDR \leftarrow (MAR); RIP \leftarrow RIP + 8$
- \*  $IR \leftarrow MDR$
- 3. \* IF  $FLAGS[ZF] == 1$  THEN
- \*  $TEMP1 \leftarrow RIP$
- \*  $TEMP2 \leftarrow IR[0:31]$
- \*  $RIP \leftarrow ALU\_OUT[ADD]$
- \* ENDIF



- b. Abilito il buffer-tri-state di RIP per scrivere sull'internal bus data, poi i dati arrivano a TEMP1 ed abilito la scrittura a questo registro : (Wtemp1=1)



- d. Prendo i 32 bit dello spiazzamento da IR[0:31], abilito il buffer-tri-state per scrivere sull'internal data bus , e lo scrivo su TEMP2 ed abilito il segnale di scrittura (Wtemp2=1).



- f. Infine alla ALU dando il codice della somma (Aluop=0000), abilito il buffer-tri-state(Ba=1) per scrivere sull'internal bus data , lo vado a scrivere nel RIP ed abilito la scrittura nel RIP(Wrip=1).

**Notiamo che si deve essere sicuri di aggiornare il registro flags se e solo se si hanno istruzioni aritmetiche.**